Phase 3 Report: Model Implementation

Graph Neural Networks for Network Intrusion Detection

Group ID: F26-10

Abdul Rehman (24L-2613) Waniya Sohail (24L-2552) Ghulam Murtaza (24L-2566)

Status of this report. Our project compares a traditional machine learning model against a graph neural network. This phase covers the **Random Forest baseline**, fully trained and evaluated, plus the data structures the graph model needs, which are built and verified. The GraphSAGE model is the immediate next step and its results are not in this report. We measured the baseline properly first, because without a trustworthy baseline the comparison this project rests on means nothing.

1. Model Selection and Justification

1.1 Random Forest (baseline)

Random Forest is the traditional machine learning half of our comparison. We picked it for four reasons.

- It is the standard baseline in published CICIDS2017 work, so our numbers can be placed next to existing results instead of sitting on their own.
- It handles our features as they are. They sit on wildly different scales, from packet counts in the tens to byte rates in the millions, and a tree splits on thresholds, so no normalisation is needed. That removes one chance of leaking information from test into training.
- It copes with heavy class imbalance through class weighting, and our data is 80.5 percent normal traffic.
- It is a genuinely strong model on tabular data, not a weak strawman. If our graph model is going to claim an improvement, the thing it improves on has to be competitive.

We trained two versions: a **binary** model (attack or not) and a **15-class** model (which specific attack). The binary model shows whether anything is detected at all; the 15-class model is the harder and more useful task, and it is where the interesting failures appear.

1.2 E-GraphSAGE (graph model, in progress)

The limitation of Random Forest is structural, not a matter of tuning. It sees one flow at a time: seventy numbers describing a single conversation, with no knowledge that the same machine opened nine hundred other connections in the same minute. Several attacks are defined precisely by that wider pattern.

We are using the **E-GraphSAGE** formulation, where each flow is an *edge* between two endpoints rather than a node. This matters because our labels are attached to flows, not to machines, so the classification has to happen on edges. GraphSAGE was chosen over GCN or GAT because it is inductive: it learns a rule for combining a neighbourhood rather than memorising fixed node positions, so it can handle IP addresses in the test set that never appeared in training. On this dataset that is not optional.

2. Implementation Details

Component	Choice
Environment	Google Colab (Python 3.13), data on Google Drive
Data handling	pandas, numpy, pyarrow (Parquet)
Baseline model	scikit-learn 1.x (RandomForestClassifier)
Graph model	PyTorch, PyTorch Geometric
Metrics	scikit-learn.metrics
Progress tracking	tqdm
Figures	matplotlib
Model persistence	joblib

We used Colab rather than a local machine because the dataset is 2.8 million rows and the graph model needs a GPU. Intermediate files are Parquet rather than CSV: four times smaller, types preserved, and loads in seconds instead of minutes.

3. Code Workflow

The pipeline splits in two after preprocessing, so both models consume exactly the same rows in two different shapes.

```
cleaned Parquet (2.81M flows, 70 features)
   ↓ per-class temporal split → train / val / test
   ■■ tabular path → feature matrix → Random Forest → per-flow prediction
   ■■ graph path → 5-minute window graphs → E-GraphSAGE → per-edge prediction
   ↓ shared evaluation (macro F1, per-class recall, confusion matrix)
```

In the graph path each five-minute window becomes its own small graph. Nodes are endpoints written as IP:port, and each flow in that window is an edge carrying the 70 features as its attributes. Node identifiers are renumbered locally per window. Windowing keeps each graph inside GPU memory and reflects how detection would actually run: on a recent slice of traffic, not the whole week at once.

4. Training Process

4.1 Data split

We split **per class, in time order**. Within each attack type, flows are sorted by timestamp; the first 70 percent is training, the next 15 percent validation, and the last 15 percent test.

Split	Flows	Share	Attack rate	Graph windows
Train	1,674,601	59.5%	19.5%	326
Validation	295,512	10.5%	19.5%	89
Test	844,344	30.0%	19.5%	175

This changes the day-based split described in Phase 2, and the reason is worth recording. CICIDS2017 runs each attack once, in one session on one day. Splitting by day therefore puts whole classes on one side: Bot, PortScan, Infiltration and the Web Attacks occur only on Thursday and Friday, so a Monday-to-Wednesday training set contains none of them. A model cannot learn a class it has never seen. We also tried random time windows, which failed differently - DDoS got zero validation flows and SQL Injection zero training flows, as short attacks fall inside one or two windows.

The per-class split guarantees every class appears in all three sets while keeping time order, so the model is always tested on later traffic than it trained on. Its honest weakness is that flows seconds apart can land either side of the boundary, making scores somewhat optimistic. We judged a slightly optimistic but complete comparison more useful than a stricter split that cannot measure most classes at all.

4.2 Hyperparameters

Setting	Value	Reason
n_estimators	100	Scores plateau beyond this; more trees cost time, not accuracy
class_weight	balanced	Rare classes would otherwise be ignored at 80.5% normal traffic
max_depth	None	Trees grow fully; the ensemble controls overfitting
n_jobs	-1	Trees are independent, so all cores are used
random_state	42	Fixed so every rerun reproduces these numbers

Trees were added in ten batches of ten using warm_start, purely so a tqdm bar could report progress during the fifteen minute fit; the resulting forest is identical to fitting 100 trees in one call. Training took 874 seconds (binary) and 890 seconds (15-class) on Colab CPU, and both models are saved with joblib. Validation is used only for checking settings - every result below comes from the test set.

5. Initial Results

5.1 Binary classification

Metric	Validation	Test
Macro F1	0.9987	0.9709
Attack recall	0.9979	0.9753

Attack precision	0.9980	0.9324
PR-AUC	0.9998	0.9861
Missed attacks	122	4,074
False alarms	117	11,645

On test the model catches 160,623 of 164,697 attack flows. The 4,074 it misses are the costly errors: a real attack reported as normal. The 11,645 false alarms are only 1.7 percent of benign traffic, but because attacks are rare they still make about seven in every hundred alerts noise. We report macro F1 rather than accuracy throughout, because predicting 'normal' for everything scores 80.5 percent accuracy while detecting nothing.

5.2 Fifteen-class classification

Macro F1 on test is **0.7663**, against 0.9849 weighted F1 and 0.9850 accuracy. The distance between those three numbers is the whole story: the common classes are handled well and the rare ones are not.

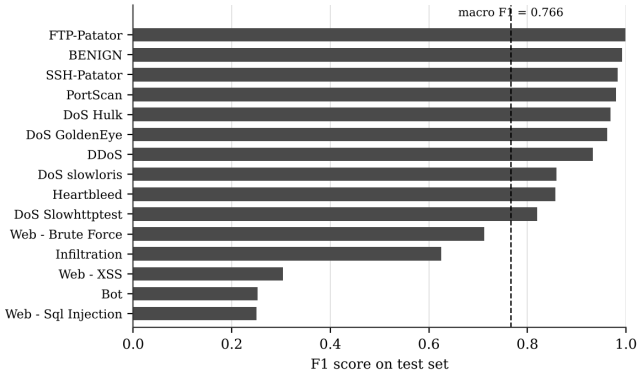


Figure 1: F1 score per class on the test set. The dashed line is the macro average, which weighs every class equally.

Attacks with a direct mechanical signature were detected reliably: FTP-Patator 0.9983, SSH-Patator 0.9831, PortScan 0.9796, DoS Hulk 0.9681, DoS GoldenEye 0.9620 and DDoS 0.9329. Port scanning, password guessing and packet floods all leave an obvious fingerprint in a single flow's packet sizes and timing, which is exactly what this model reads.

Three classes failed, and they failed in different ways.

- **Bot (F1 0.2528).** The model found 91 of 587 bot flows and labelled 496 of them as normal traffic. It is not confusing bots with another attack; it cannot see them at all. A single bot flow is a small, regular connection to an ordinary port, and nothing in the 70 features separates it from browsing.
- **Web Attack XSS (0.3035) and Brute Force (0.7122).** These two swap with each other, 124 flows one way and 112 the other. Both are HTTP requests to the same server, on the same port, within the same hour. At flow level they are close to indistinguishable.
- **Heartbleed, Infiltration, SQL Injection.** 4, 11 and 7 test flows respectively. One flow moves the score by a quarter, so these values are noise rather than measurements. We report them for completeness and draw no conclusion from them.

5.3 Validation against test

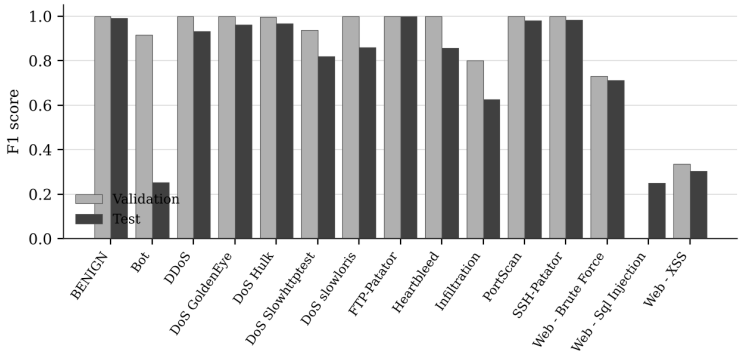


Figure 2: the same model on validation and on test. Most classes hold; Bot collapses from 0.9163 to 0.2528.

Macro F1 falls from 0.8476 on validation to 0.7663 on test. Validation flows sit immediately after training in time, often seconds apart and from the same attack burst, so the model is recognising something it has nearly already seen; test flows come from a later stretch. The drop is the expected result - had the two matched, we would suspect leakage. The classes that fall are those whose behaviour drifts: Bot most of all, then DoS slowloris (0.9992 to 0.8585) and Slowhttptest (0.9370 to 0.8200), both attacks that hold connections open, where timing depends on network conditions and learned thresholds stop fitting.

5.4 What this implies for the graph model

The weakest classes are all defined by communication pattern rather than by any one flow's contents. A bot is one machine contacting a control server repeatedly; infiltration is movement from one compromised machine into others. These are relationships between hosts, and a model reading flows in isolation cannot access them. That is the specific claim our graph model is built to test, and Bot recall is the number we will watch. If GraphSAGE does not improve there, that is a result worth reporting too.

6. Challenges Faced

- **Choosing the split took three attempts.** Section 4.1. The day-based split left four classes with no training examples; a random window split left DDoS with no validation data. We now run a per-class count of every split as a standing check.
- **The graph files were built against the old split.** Created in Phase 2, before validation existed, their training set silently held what is now training plus validation. Unchecked, the two models would have trained on different data and the comparison would have been meaningless. We rebuilt all three sets from the current split column and verified edge counts match row counts exactly.
- **No progress visibility during training.** scikit-learn fits a forest silently, so a fifteen minute run is indistinguishable from a hang. We added trees in batches under tqdm. scikit-learn warns that balanced class weights with warm_start can be unsafe, which applies when the data changes between batches; ours does not, so the weights are identical each round.
- **No GPU, and limited memory.** Colab gave us a CPU-only runtime, tolerable for Random Forest but not for the graph model, and switching runtimes clears all variables - so every intermediate result is written to Drive first. Holding the dataframe and all three graph sets at once also exceeds Colab RAM, so the rebuild constructs one split, saves it, frees it, and moves on.
- **Classes too small to evaluate.** No split can give Heartbleed a meaningful test set when the dataset contains eleven of them. That is a property of CICIDS2017, not something we can engineer around, and we state it rather than reporting the figures as if they were measurements.

7. Current Position and Next Steps

The Random Forest baseline is complete, evaluated and saved: test macro F1 0.9709 binary and 0.7663 across fifteen classes. Those are the numbers the graph model must beat. The window graphs are rebuilt against the current split and verified edge-for-edge against the tabular data, so both models consume identical rows.

Remaining: implement the E-GraphSAGE edge classifier, train it on both tasks, and evaluate with the same metrics. For the final phase we also plan to repeat training across several seeds with means and standard deviations, and to run a shuffled-edge ablation that destroys graph structure while keeping the features - testing whether any improvement comes from structure rather than extra model capacity.

Files. F26-10.ipynb, rf_binary.joblib and rf_multiclass.joblib (trained models), train/val/test_5min_v2.pkl (window graphs), cicids2017_v2.parquet (split dataset). Repository: https://github.com/Ab647-blip/Network-Intrusion-Detection